

Python Programming with Artificial Intelligence

Introduction

Chengwei QIN

Assistant Professor @AI Thrust



1. Course Information
2. Introduction to Python
3. Python and IDE Setup
4. GitHub Copilot Setup
5. Simple LLM API Call demo
6. Python Basics



Course Information

Class Schedule: Sep 1, 2026 – Dec 7, 2026

- Time: Mon, 04:30PM - 05:50PM **and** Fri 12:00PM - 01:20PM
- Location: Lecture Hall A

Instructor: Chengwei QIN

TA:

- Hehai LIN, hlin709@connect.hkust-gz.edu.cn
- Jiayu YANG, jyang729@connect.hkust-gz.edu.cn

Others to be determined.



Chengwei QIN

LLM Reasoning

Agents

Multimodality

Efficient Learning

- Assistant Professor@AI Thrust, working on NLP and LLMs
- Bachelor's degree from THU, PhD degree from NTU

Paper Awards

- WAIC Youth Outstanding Paper Award Shortlist, 2026 (LLM Reasoning)
- EMNLP Outstanding Paper Award, 2024 (LLM Reasoning)
- ACL Area Chair Award, 2023 (Multimodality)

Other Experience

- Meta AI, AWS AI

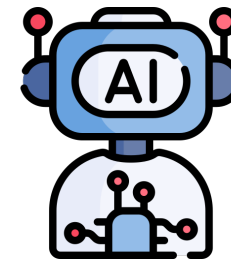
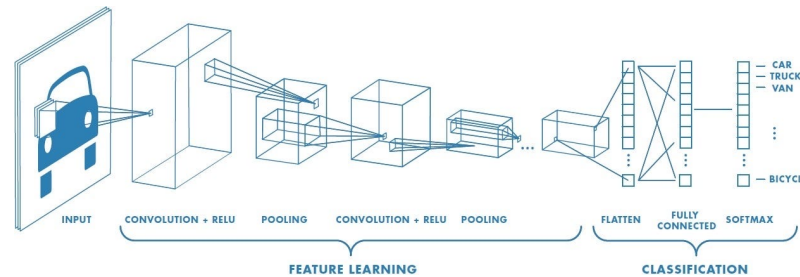
Chengwei QIN, Email: chengweiqin@hkust-gz.edu.cn

Office Hour: Friday, 1:30pm - 2:30pm, W3-310 or by appointment

Course Goals

By the end of this course, students will be able to:

- Develop a strong foundation in advanced Python programming concepts
- Learn to implement core Machine Learning and Deep Learning models using Python
- Gain proficiency in building Python-based AI applications by integrating models, data pipelines, APIs, and evaluation workflows
- Acquire hands-on experience with modern AI-assisted software development tools and learn how to use them effectively for project development



Course Content Overview

Week	Title
1 ~ 2	Python Fundamentals
2 ~ 4	Python Packages for Data Science and Machine Learning with scikit-learn
4 ~ 6	Deep Learning (e.g., visual models and language models including LLMs)
7	AI Agent.
8	Embodied AI.
9	Coding Agent.
10 ~ 12	Software Engineering with AI
13	Final Project Presentation - presentations, reflection, wrap-up.

Grading Scheme

Course Component	Weight (%)
Attendance	10%
In-Class Practices	20%
Assignment (3 times)	30% (10% each)
Final Group Project	40%

In the first two weeks, no attendance will be taken and no in-class practice will be graded.

Attendance (Starting from Sep. 15, 2026, after the Add/Drop Period)

- Attendance will be recorded **every** class.

In-class Practices (Starting from Sep. 15, 2026, after the Add/Drop Period)

- Each class will include some short practice exercises
- 1-2 practice per class will count toward to your grade
- Make sure to complete and submit these graded practices

Everyone has **2 free absences** for attendance, and **2 free absences** for in-class practice.

This is a group project. Each team should consist of **2-4 students**.

- **Design and build** an interesting and creative system
- **Write a report** that explains your design and findings
- **Present in class:** share your system design, give a demo, and highlight your contributions
- **Submit your code** for evaluation and reproducibility

Deadline -- 23:59 on the day (Dec. 3 for L01) right before Week 13's first class
A suggested list of final project topics will be provided in the coming weeks.

Overall Assessment

Cumulative Score (out of 100)	Possible Grades
90~100	A-/A/A+
70~90	B-/B/B+/A-
50~70	C-/C/C+/B-
40~50	D/C-
<40	F

Note:

1. The assessment primarily focuses on **how well the expected learning outcomes are achieved**.
2. The **mapping** provided above is from past experiences and should be considered as **a preliminary guide**.

■ ■ ■ Important Notes

- Do not plagiarize. Academic honesty is essential. Any copied work will result in serious consequences.
- Teaching Style: ~60% Lectures w/ Concepts/Examples + ~40% Practices

- We are open-minded to Generative AI tools.
- This is a fundamental programming course, and one of the important purpose is for you to develop your own coding skills
- Understanding the underlying principles helps you communicate better with AI and enables more effective use of AI tools.
- Remember: AI should assist, not replace your thinking

“What I cannot create, I do not understand.”
---Richard Feynman



More Questions?



Introduction to Python

One of the most popular programming language

<https://www.tiobe.com/tiobe-index/>

The TIOBE Programming Community index is an indicator of the popularity of programming languages.

Aug 2026	Aug 2025	Change	Programming Language		Ratings	Change
1	1			Python	18.53%	-7.61%
2	3	▲		C	11.10%	+2.07%
3	2	▼		C++	8.62%	-0.56%
4	4			Java	8.25%	-0.34%
5	5			C#	4.09%	-1.43%
6	6			JavaScript	2.63%	-0.52%
7	7			Visual Basic	2.18%	-0.15%
8	12	▲▲		SQL	1.88%	+0.16%
9	14	▲▲		R	1.56%	+0.19%
10	18	▲▲		Rust	1.45%	+0.31%

Beginner-friendly, entry-level language that is originally designed for education

- **High Readability**
 - Keywords that meaningful in English
 - New lines to finish a command and indentation to define the scope
- **Quick development and experimentation**
 - Vast Libraries Support: over 600,000 packages in PyPI
 - High-level built-in functionalities (implemented in C)
 - Script language with interactive mode for fast testing and debugging
- **Cross-platform compatibility:** works seamlessly on macOS, Linux, windows, and even Raspberry Pi
- **Large and active community:** abundant tutorials, resources, and support

Sacrifices efficiency for simplicity

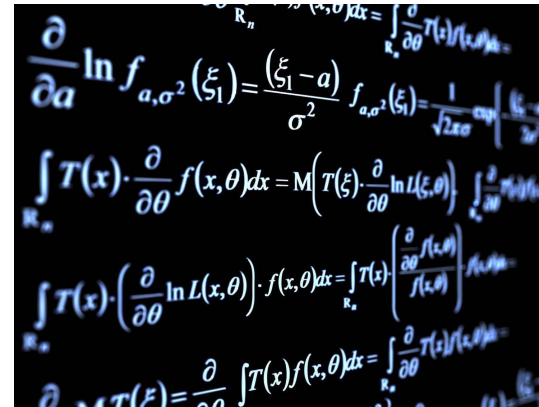
- **Slower execution:** Basic operations run less efficiently
- **Higher memory usage:** Not ideal when memory optimization is critical (e.g., mobile apps)

Solution: Implement efficiency-critical components in C/C++ and expose them through a Python interface

Python is suitable for ...

Anything not requiring efficiency for practical deployment

- **Research**
 - Data Process, Data Analysis, Calculation, ...
- **Automation Script**
 - OS, Database, ...
- **Prototype product development**
- **Machine Learning/Deep Learning**
- **Web Crawling**


$$\frac{\partial}{\partial a} \ln f_{a, \sigma^2}(\xi_1) = \frac{(\xi_1 - a)}{\sigma^2} f_{a, \sigma^2}(\xi_1) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\xi_1 - a)^2}{2\sigma^2}\right)$$
$$\int_{\mathcal{R}_n} T(x) \cdot \frac{\partial}{\partial \theta} f(x, \theta) dx = M\left(T(\xi) \cdot \frac{\partial}{\partial \theta} \ln L(\xi, \theta)\right) \int_{\mathcal{R}_n} T(x) f(x, \theta) dx$$
$$\int_{\mathcal{R}_n} T(x) \cdot \left(\frac{\partial}{\partial \theta} \ln L(x, \theta)\right) \cdot f(x, \theta) dx = \int_{\mathcal{R}_n} T(x) \cdot \left(\frac{\partial}{\partial \theta} \frac{f(x, \theta)}{f(x, \theta)}\right) f(x, \theta) dx$$
$$\frac{\partial}{\partial \theta} \int_{\mathcal{R}_n} T(x) f(x, \theta) dx = \int_{\mathcal{R}_n} T(x) \frac{\partial}{\partial \theta} f(x, \theta) dx$$





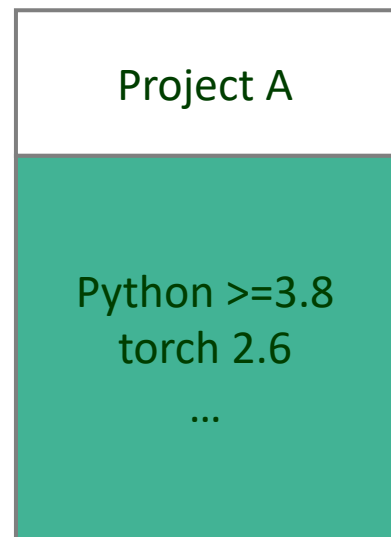
Python and IDE Setup

Since you are already familiar with coding (or even Python), we'll skip some basic content and go over this part quickly. Please read this part by your self after class if needed, and feel free to ask me if you have difficulty with the setup. I will help.

When you install **Anaconda** (Anaconda: <https://www.anaconda.com/>), you get:

- **Python** — the Python interpreter for running Python programs.
- **Many popular Python packages** — such as NumPy, pandas, and Matplotlib.
- **Conda** — a tool for creating and managing multiple isolated Python environments, helping avoid conflicts between different Python versions and packages.

<https://docs.conda.io/projects/conda/en/latest/user-guide/install/index.html>



Conflict?

Conda Commands

After installing anaconda, to open the prompt window:

- Windows: Search for Anaconda Prompt
- macOS: Open Terminal
- Linux: Open Terminal

In Prompt window:

- Create an environment
- Activate the environment
- Install packages
- Remove packages
- Remove an environment

```
# create an environment
```

```
conda create -n env1 python=3.6
```

```
# switch the environment
```

```
conda activate -n env1
```

```
# install packages
```

```
conda install seaborn
```

```
# remove an environment
```

```
conda remove -n env1 --all
```

Example: Do some analysis for Fibonacci sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, ...
 - $a[n] = a[n-1] + a[n-2]$
- Analysis
 - `print a[1000000000]`

Interactive vs. Script Mode

Example: Do some analysis for Fibonacci sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, ...
 - $a[n] = a[n-1] + a[n-2]$
- Analysis
 - `print a[1000000000]`

```
4  lst = [0, 1]
5  for u in range(1, 1000000001):
6      lst.append((lst[u-1] + lst[u-2]) % 10000000000)
7
8  print(lst[1000000000])
```

31s to execute the code

Interactive vs. Script Mode

Example: Do some analysis for Fibonacci sequence

- Fibonacci sequence: 1, 1, 2, 3, 5, 8, ...

➤ $a[n] = a[n-1] + a[n-2]$

- Analysis

➤ `print a[1000000000]`

➤ `Print a[999999999] + a[1002]`

```
4  lst = [0, 1]
5  for u in range(1, 1000000001):
6      lst.append((lst[u-1] + lst[u-2]) % 10000000000)
7
8  print(lst[1000000000])
9  print(lst[999999999] + lst[1002])
```

Another 31s to execute the code

Interactive mode can save time

```
Python 3.9.13 (main, Aug 25 2022, 18:29:29)
[Clang 12.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> lst = [0, 1]
>>> for u in range(2, 1000000001):
...     lst.append((lst[u-1] + lst[u-2]) % 10000000000)
...
>>> print(lst[1000000000])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
>>> print(lst[100000000])
7760546875
>>> print(lst[99999])
6278790626
>>> print(lst[99999] + lst[1002])
8451423002
>>> print(lst[9999999] * lst[10000])
7364937187632913750
>>>
```

31s for processing
Then any analysis

How to reuse the code
next time?

Interactive vs. Script Mode

Interactive	File
Instant feedback	No instant feedback
Run one line at a time	Execute the whole file
Not reusable	Reusable
Difficult to debug	Easy to debug

Observation:

- Interactive mode is quick and convenient for testing ideas, but not suitable for larger projects.
- Script mode is structured and reusable, but less flexible for immediate feedback

Motivation:

Can we combine the advantages of both?

This leads us to Jupyter Notebook.

Hybrid Approach: combine script and interactive modes by coding in blocks

Time consuming
data processing

Interactive analysis

Adding analysis without re-
executing the processing

```
jupyter_example.ipynb > print(lst[9])
+ 代码 + Markdown | ▶ 全部运行 | 清除所有单元格输出 | Outline ...
py37 (Python 3.7.13)
```

```
[1]: lst = []
    for i in range(100000000):
        lst.append(i)
    print(len(lst))
... 100000000
```

```
[3]: print(lst[99])
... 99
```

```
[4]: print(lst[9])
... 9
```

✓ Interactive Computing

✓ Reusable Scripts

Getting started with Jupyter Notebook

Install Jupyter Notebook

Shell

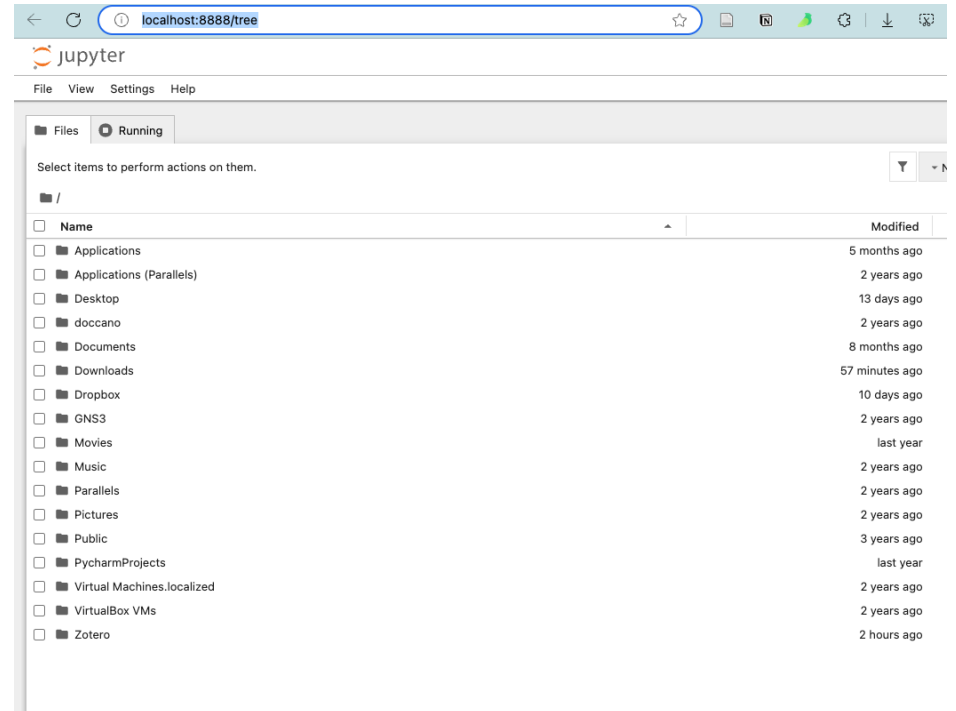
```
$ pip install jupyter
```

Run Jupyter Notebook

```
jupyter-notebook
[I 2025-08-26 16:39:25.790 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2025-08-26 16:39:25.791 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2025-08-26 16:39:25.793 ServerApp] jupyterlab | extension was successfully linked.
[I 2025-08-26 16:39:25.794 ServerApp] notebook | extension was successfully linked.
[I 2025-08-26 16:39:25.943 ServerApp] notebook_shim | extension was successfully linked.
[I 2025-08-26 16:39:25.963 ServerApp] notebook_shim | extension was successfully loaded.
[I 2025-08-26 16:39:25.964 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2025-08-26 16:39:25.965 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2025-08-26 16:39:25.967 LabApp] JupyterLab extension loaded from /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/jupyterlab
[I 2025-08-26 16:39:25.967 LabApp] JupyterLab application directory is /Library/Frameworks/Python.framework/Versions/3.13/share/jupyter/lab
[I 2025-08-26 16:39:25.967 LabApp] Extension Manager is 'pypi'.
[I 2025-08-26 16:39:25.996 ServerApp] jupyterlab | extension was successfully loaded.
[I 2025-08-26 16:39:25.998 ServerApp] notebook | extension was successfully loaded.
[I 2025-08-26 16:39:25.999 ServerApp] Serving notebooks from local directory: /Users/chenhuangxun
[I 2025-08-26 16:39:25.999 ServerApp] Jupyter Server 2.17.0 is running at:
[I 2025-08-26 16:39:25.999 ServerApp] http://localhost:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:25.999 ServerApp] http://127.0.0.1:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:25.999 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2025-08-26 16:39:26.003 ServerApp]

To access the server, open this file in a browser:
file:///Users/chenhuangxun/Library/Jupyter/runtime/jpserver-76649-open.html
Or copy and paste one of these URLs:
http://localhost:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
http://127.0.0.1:8888/tree?token=cba55be6152351d9135fe716f3e65856371d91ab2ce3a285
[I 2025-08-26 16:39:26.012 ServerApp] Skipped non-installed server(s): bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-langserver, jedi-language-server, julia-language-server, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yaml-language-server
```

Open Jupyter in a web-browser

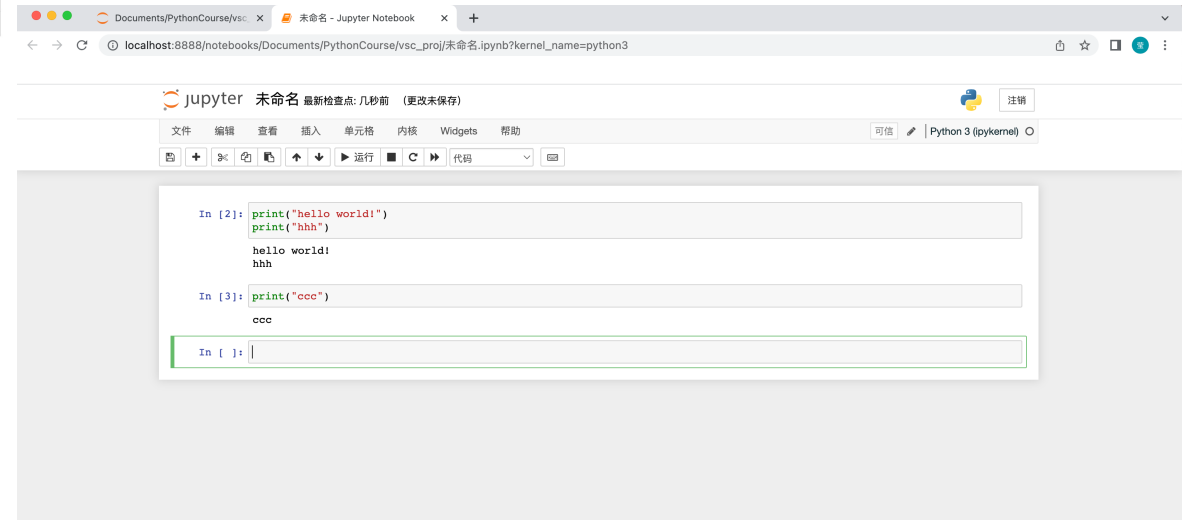
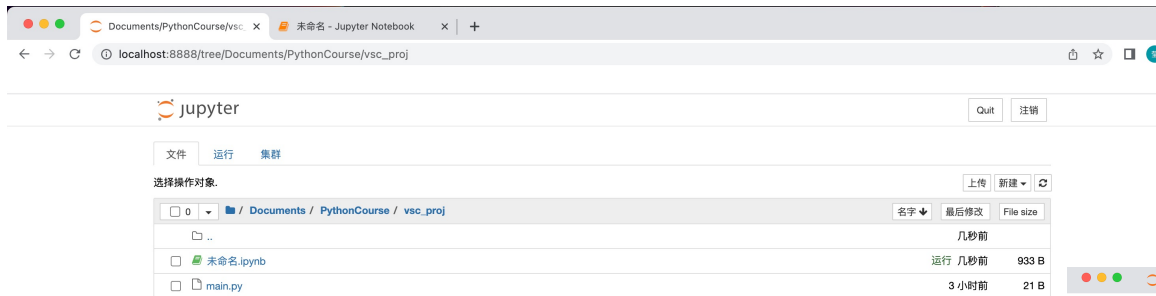


Getting started with Jupyter Notebook

Create an ipynb file and try

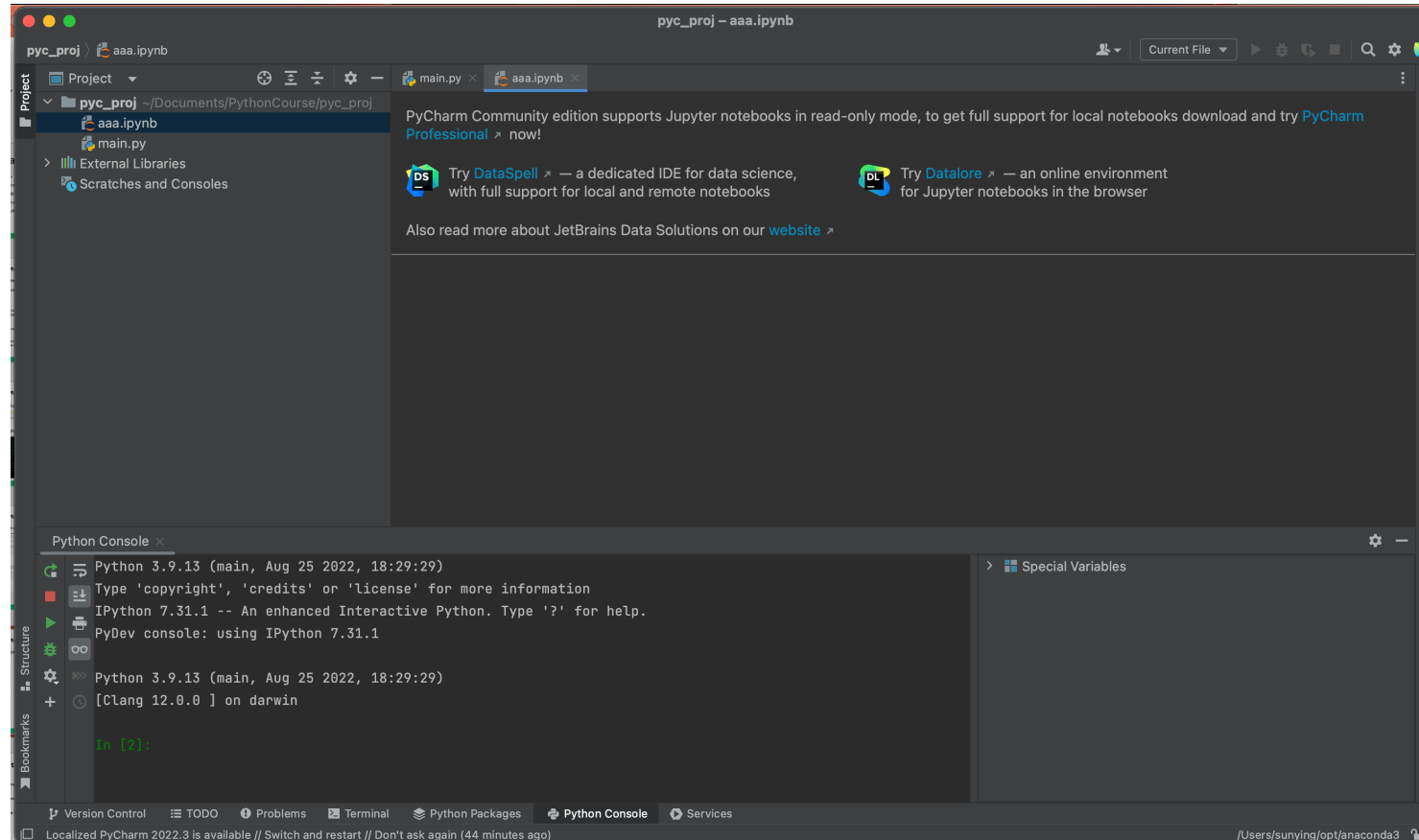
Write and run code in individual cells

View the results instantly within the browser



Jupyter Notebook (PyCharm)

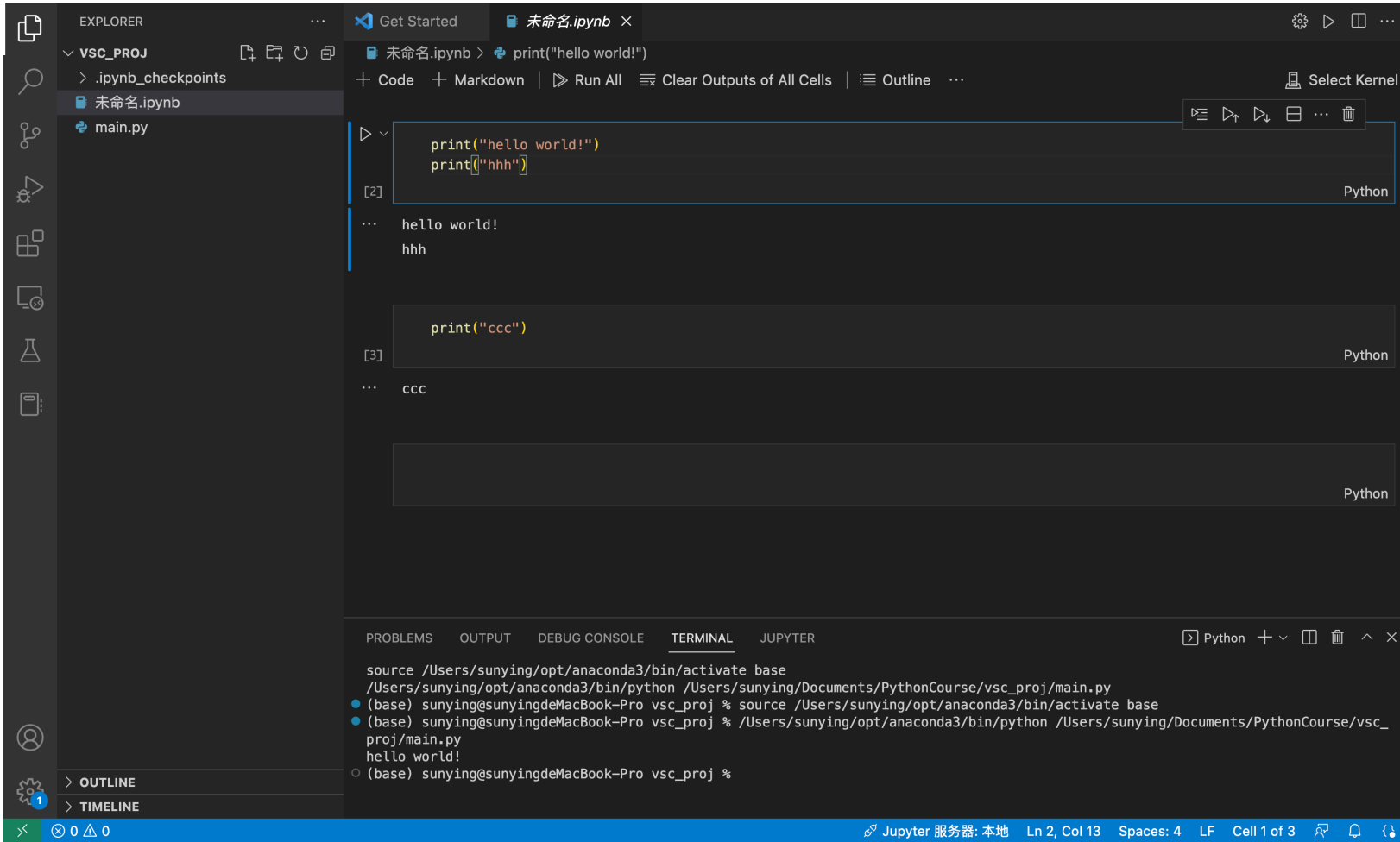
Just create the ipynb file



only for professional

Jupyter Notebook (VScode)

Just create the ipynb file



The screenshot displays the VS Code interface with a Jupyter Notebook open. The Explorer sidebar on the left shows a project named 'VSC_PROJ' containing a file named 'main.py'. The Jupyter Notebook editor shows three cells:

- Cell [2]:

```
print("hello world!")
print("hhh")
```

Output:

```
hello world!
hhh
```
- Cell [3]:

```
print("ccc")
```

Output:

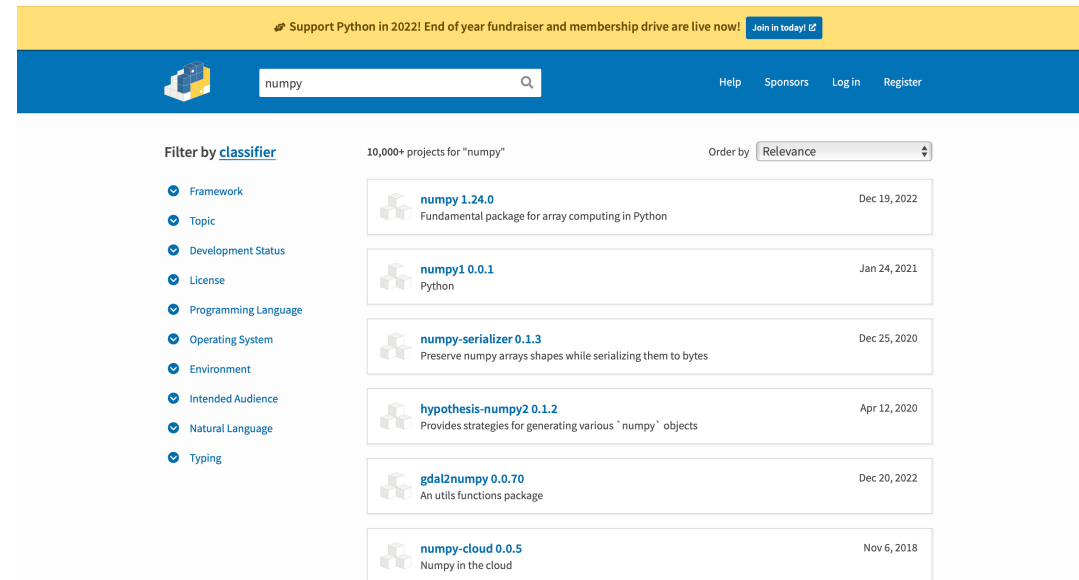
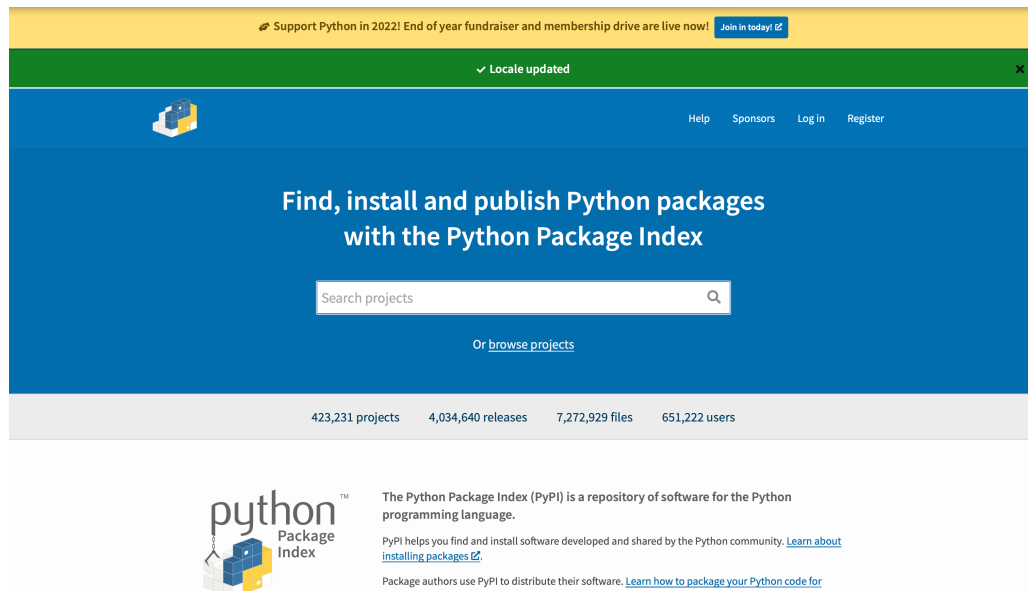
```
ccc
```
- Cell [4]: (Empty cell)

The terminal at the bottom shows the command to run the notebook:

```
source /Users/sunying/opt/anaconda3/bin/activate base
/Users/sunying/opt/anaconda3/bin/python /Users/sunying/Documents/PythonCourse/vsc_proj/main.py
(base) sunying@sunyingdeMacBook-Pro vsc_proj % source /Users/sunying/opt/anaconda3/bin/activate base
(base) sunying@sunyingdeMacBook-Pro vsc_proj % /Users/sunying/opt/anaconda3/bin/python /Users/sunying/Documents/PythonCourse/vsc_proj/main.py
hello world!
(base) sunying@sunyingdeMacBook-Pro vsc_proj %
```

PyPI - The Python Package Index

The Python Package Index (PyPI) is the official third-party software repository for the Python programming language. It serves as **a central hub where Python developers can publish and share their packages and libraries** with the wider community.



<https://pypi.org>

PyPI - The Python Package Index

Install and use packages in PyPI: pip (installed with Python)

Install: run `pip install <package-name>`

```

pip3 --version
pip 25.2 from /Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/pip (python 3.13)

python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    import numpy
ModuleNotFoundError: No module named 'numpy'
>>> exit()

pip3 install numpy
Looking in indexes: https://mirrors.aliyun.com/pypi/simple/
Collecting numpy
  Downloading https://mirrors.aliyun.com/pypi/packages/78/45/d4698c182895af189c463fc91d70805d455a227261d950e4e0f1310c2550/numpy-2.3.2-cp313-cp313-macosx_14_0_arm64.whl (5.1 MB)
    5.1/5.1 MB 6.5 MB/s 0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.3.2

python3
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy
>>>
```



GitHub Copilot Setup

- Copilot provides AI-powered code suggestions and chat assistance inside coding environments.
- It can help with code completion, code explanation, debugging, test generation, and documentation.
- Copilot helps you write code, but you are responsible for understanding, testing, and validating the code.

Github Copilot General Setup Workflow

1. Create or sign in to a GitHub account.
2. Apply for GitHub Education / Copilot Student access.
3. Install an IDE or notebook environment. (Most of you have already done)
4. Install the GitHub Copilot extension/plugin.
5. Sign in with GitHub in the IDE.
6. Test inline suggestions and Copilot Chat.

Github Copilot Registration

- Go to GitHub Education Student benefits. <https://github.com/education/students>
- Sign in with your GitHub account.
- Verify your student status using your university email or required documents.
- After approval*, enable GitHub Copilot Student access.
- Use the same GitHub account to sign in from VS Code or PyCharm.

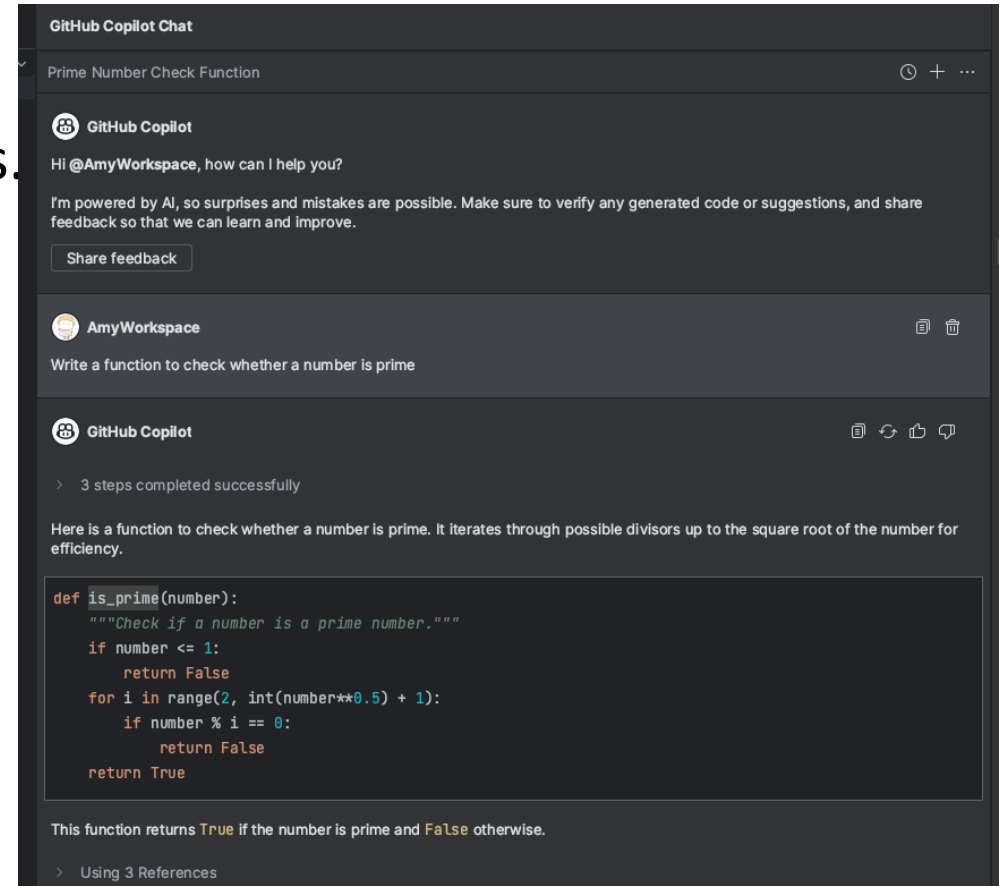
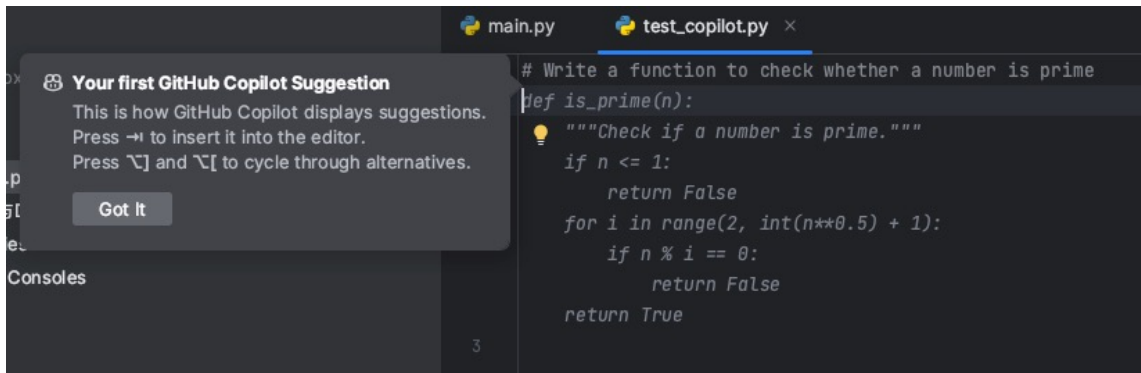
* Approval may take time.

* There is also free plan to get started.

A Chinese tutorial: https://blog.csdn.net/PGJ_168/article/details/150267222

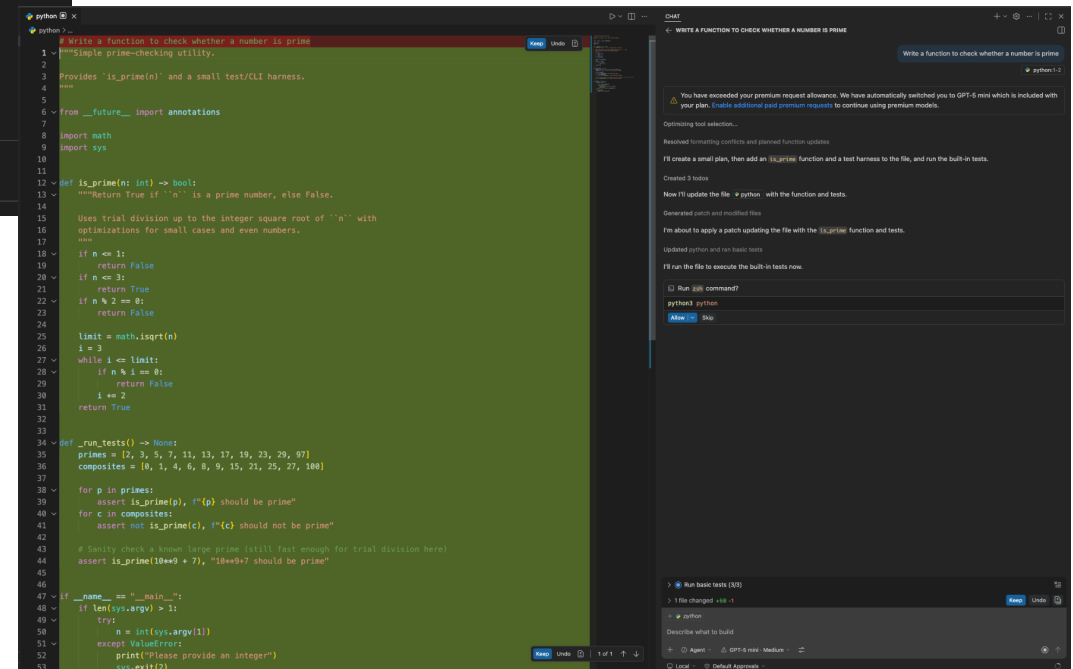
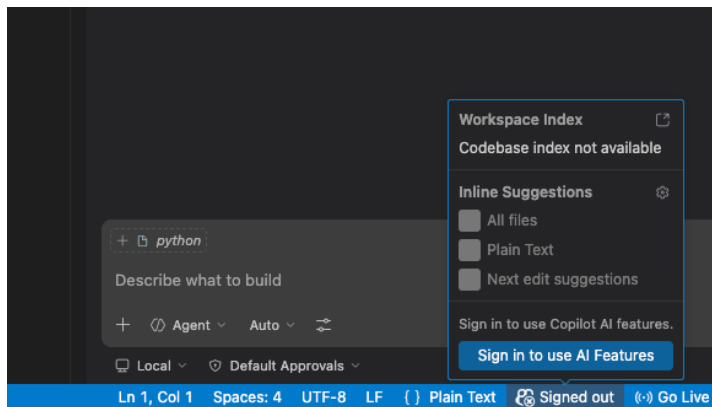
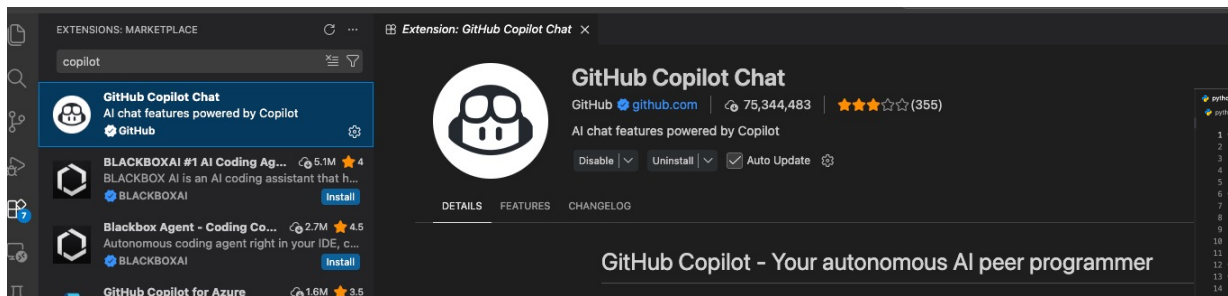
Github Copilot + PyCharm

- Setup PyCharm first
- Go to Settings / Preferences → Plugins, and install the Copilot plugin.
- Restart PyCharm if required.
- Sign in with your GitHub account.
- Open a Python file and test copilot suggestions.



Github Copilot + VS Code

- Setup VS code first
- Install the GitHub Copilot extension.
- Sign in with your GitHub account.
- Open a .py file and test Copilot suggestions.



- NOT support browser-only Jupyter Notebook
- Recommended method: use Jupyter Notebook inside VS Code/PyCharm
- Open or create a .ipynb file in IDE.
- Sign in to GitHub.
- Try Copilot suggestions inside notebook code cells.

Allowed

- Use Copilot to understand examples.
- Use Copilot to get coding hints.
- Use Copilot to debug error messages.
- Use Copilot to generate possible test cases.
- Use Copilot to improve readability after you understand the code.

Not Allowed

- Submit code you cannot explain.
- Submit AI-generated code without testing.
- Use Copilot to bypass independent work.
- Copy complete solutions without understanding.

You may use AI tools as assistants, but **you are fully responsible for your submitted code.**



Simple LLM API Call demo

Why Call an LLM API?

API call is the basic building block of AI applications and agents. LLM API helps our code use AI capabilities.

Relationship with Copilot:

Direct LLM API call: you write the request yourself.

Copilot: the IDE/plugin writes and manages the request for you.

What Is an LLM API Call?

Your Python program sends a request to the LLM server.

The request includes:

- API URL, API key, model name
- system prompt, user prompt
- some parameters, e.g., temperature

The server returns a response, usually in JSON format.

A Minimal Example

```
import requests
import json

url = "https://aigc-api.hkust-gz.edu.cn/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": api_key
}

data = {
    "model": "DeepSeek-V4-Pro",
    "messages": [
        {"role": "system", "content": "You are a helpful coding assistant."},
        {"role": "user", "content": "Write a Python program to print all prime numbers between 0 and 100."}
    ],
    "temperature": 0.7
}

response = requests.post(url, headers=headers, data=json.dumps(data))
```

The API key should not be hard-coded in the source code.

A Minimal Example: Reading API Key from a File

```
def load_api_key(key_file):  
    with open(key_file, "r", encoding="utf-8") as f:  
        return f.read().strip()  
  
api_key = load_api_key("api_key.txt")
```

- Store the key in api_key.txt.
- Do not share the key.
- Do not upload the key to GitHub.
- Add api_key.txt to .gitignore.

A Minimal Example: Parse and Use the Response

```
response_json = response.json()
message = response_json["choices"][0]["message"]["content"]
print(message)
```

Calling an LLM API is not only about getting text.

It is the first step toward building AI-powered Python applications.

Do not outsource your thinking too early.

1. Think first

Understand the problem, design your own solution, and write your first version of the code.

2. Then compare with AI

Ask AI to solve the same problem and compare its solution with yours.

3. Ask why

Do not only ask AI for the answer. Ask:

- Why does this solution work?
- What is different from my solution?
- Which implementation is more readable or efficient?
- Are there any hidden bugs or edge cases? Etc.

4. Use AI as a learning partner

AI can explain, debug, suggest alternatives, and generate tests — but you are responsible for understanding and validating the code.

Your programming mental muscle needs training. Use AI to strengthen it, not to replace it.

- Course Information
- Introduction to Python
- Python and IDE Setup
- GitHub Copilot Setup
- Simple LLM API Call demo

If you decide to continue with this course, please make sure that, before the next lecture, you are familiar with Python environment, have chosen your preferred IDE, set up Copilot, and mastered the basic LLM API call.

Python Programming with Artificial Intelligence

Thank you for listening!

